SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO5**	**Assessment:**	**ASSESSMENT TOOL1- Constraint-Based Problem Solving**
Weightage:	**40%**	**Total Marks:**	**25**
CO5 Statement:	*Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.*		
Student Name:	**ANUSHYA DEVI V**	**Reg.No.:**	**192424013**
Department:		**Date:**	

ANNEXURE A

Constraint-Based Problem Solving

1.Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2.Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3.Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4.Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5.Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μs
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct:

I ANUSHYA DEVI V (192424013) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Constraint-Based Problem Solving

Given Constraints

- CPU utilization: must be below 15%
- Sensors: multiple sensors generate interrupts every 1 ms
- Interrupt handling: vectored interrupts
- Goal: minimum I/O communication latency

Proposed I/O Communication Mechanism

1. Select Memory-Mapped I/O

Memory-mapped I/O is preferred because the CPU can access sensor registers using normal load/store instructions. It generally gives lower latency and simpler programming than separate I/O instructions.

2. Use Vectored Interrupts

Each sensor is assigned a separate interrupt vector.

Example:

Sensor 1 → Interrupt Vector 1 ─┐
Sensor 2 → Interrupt Vector 2 ─┤
Sensor 3 → Interrupt Vector 3 ─┤ → CPU → ISR → Read Sensor Register
Sensor 4 → Interrupt Vector 4 ─┘

When a sensor generates an interrupt, the CPU directly jumps to its corresponding ISR instead of checking all sensors.

CPU Utilization

If each interrupt requires approximately 10 μs of CPU processing:

- Interrupt frequency = 1 / 1 ms = 1000 interrupts/second
- CPU time = 1000 × 10 μs = 10 ms/second
- CPU utilization = 1%

Even with several sensors, the ISR should be kept short and efficient so that total utilization remains below 15%.

Final Selection

Requirement	Design Choice
Low latency	Memory-Mapped I/O
Fast interrupt identification	Vectored Interrupts
Sensor rate	1 ms period
ISR	Short and optimized
CPU utilization	< 15%
Data transfer	Read sensor register → buffer → main task

Conclusion

Memory-mapped I/O with vectored interrupts is the best choice for this real-time embedded system. Memory-mapped I/O provides fast register access, while vectored interrupts allow the CPU to immediately service the correct sensor. Keeping ISRs short ensures that CPU utilization stays below 15% and real-time response is maintained.

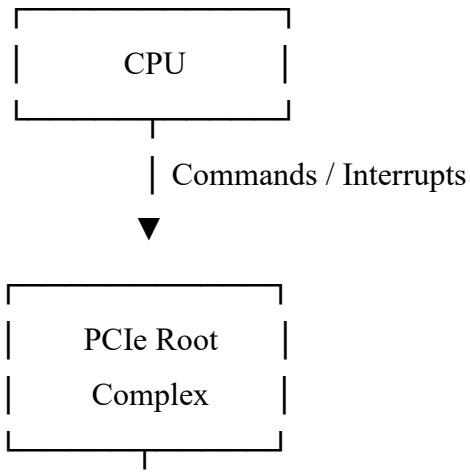
2. Constraint-Based I/O Subsystem Design

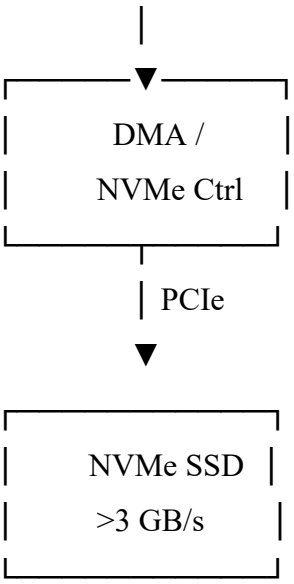
Given Constraints

- Storage throughput: > 3 GB/s
- Interface: NVMe over PCIe
- CPU: should not handle bulk data transfer
- Requirement: DMA + interrupt notification

Proposed Design

Use an NVMe SSD connected through PCIe, with a DMA-based data transfer mechanism.

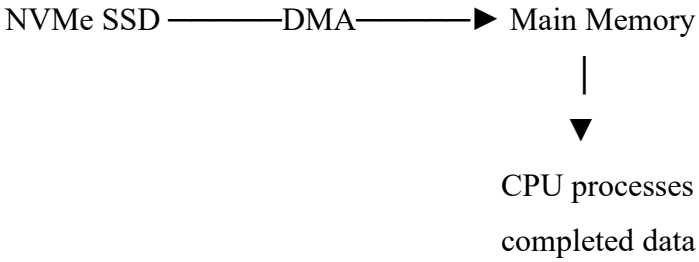




Working

- CPU sends an NVMe command to the NVMe controller.
- The CPU provides the memory address and transfer information.
- The DMA controller transfers data directly between the NVMe SSD and main memory.
- The CPU does not copy the bulk data, reducing CPU workload.
- After the transfer is completed, the NVMe controller generates an interrupt.
- The CPU handles the interrupt and processes the completed I/O request.

DMA Operation



H ow the Constraints Are Satisfied

Constraint	**Solution**
> 3 GB/s throughput	High-speed PCIe NVMe
Must use NVMe over PCIe	NVMe SSD connected through PCIe
CPU should not transfer bulk data	DMA performs data movement
Notification required	Interrupt after transfer
High performance	Queue-based NVMe commands + DMA

Conclusion

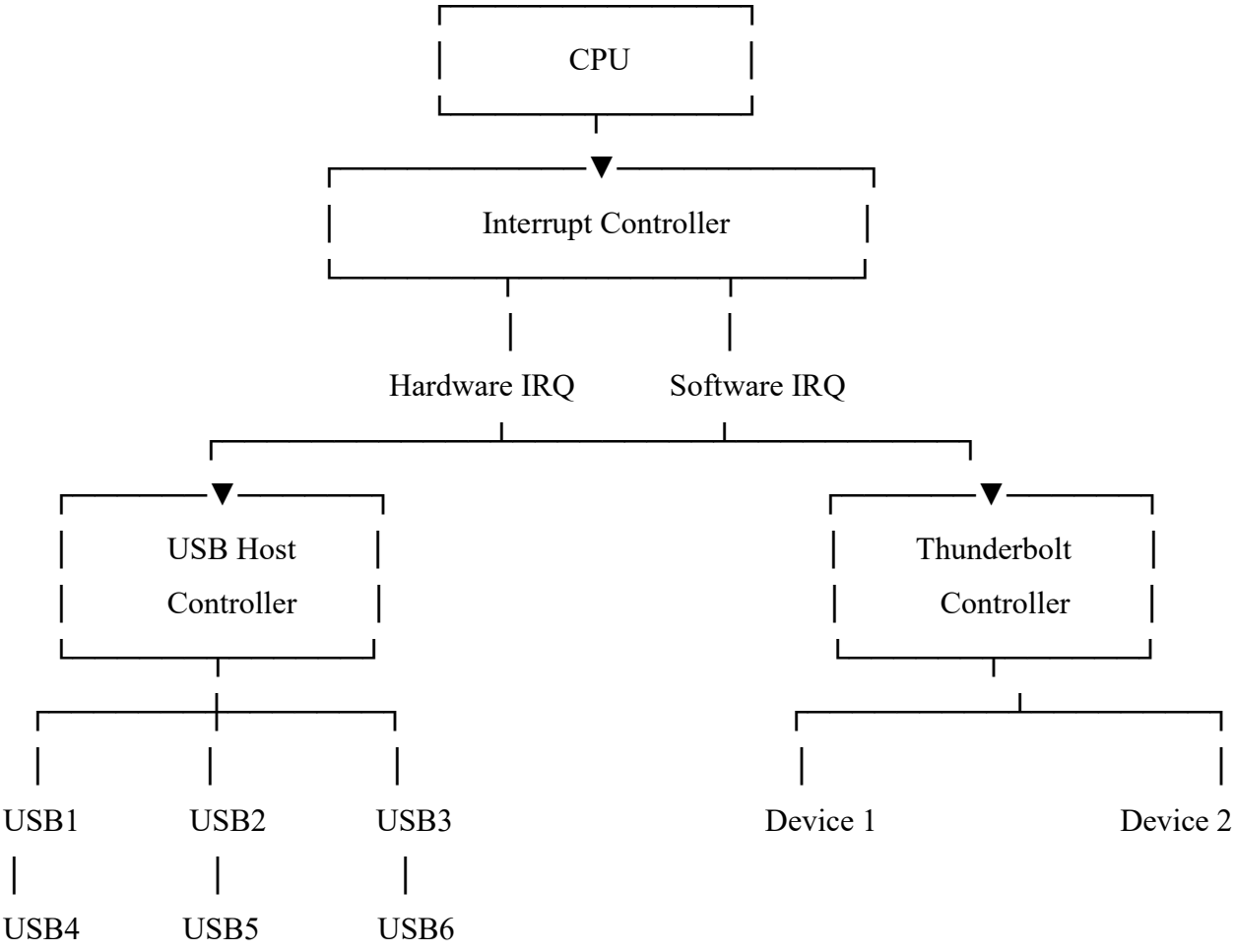
The optimal design is NVMe SSD + PCIe + DMA + interrupt notification. The CPU only manages commands and completion events, while DMA performs the bulk data transfer directly between the NVMe device and main memory. This reduces CPU overhead and supports the required >3 GB/s throughput.

3. Multi-Device Communication Architecture

Given Constraints

- 6 peripherals: USB
- 2 high-speed devices: Thunderbolt
- Avoid interrupt starvation
- Use both hardware and software interrupts
- Ensure fair bus arbitration

Proposed Architecture



Working

- **USB Devices**: Six USB peripherals are connected through a USB host controller. The controller manages communication and generates hardware interrupts when a device requires CPU attention.
- **Thunderbolt Devices:** Two high-speed devices are connected through a Thunderbolt controller. Since Thunderbolt provides high bandwidth, it receives efficient bus scheduling to prevent delays.
- **Hardware Interrupts:** Hardware interrupts are generated by USB and Thunderbolt controllers when I/O events occur. Example: USB Device → Hardware Interrupt → Interrupt Controller → CPU
- **Software Interrupts:** Software interrupts are used by the operating system for controlled I/O services, driver requests, and deferred processing. Application → System Call/Software Interrupt → OS → Device Driver

Avoiding Interrupt Starvation

Use a priority-based interrupt controller with round-robin handling among devices of the same priority.

- High-priority events are handled quickly.
- Lower-priority devices are not ignored indefinitely.
- Interrupts can be queued when multiple devices request service.
- Long interrupt routines should be avoided.

Fair Bus Arbitration

Use round-robin arbitration:

USB1 → USB2 → USB3 → USB4 → USB5 → USB6

↑ ↓

└──────────── Thunderbolt 1/2 ────────┘

Each device gets a fair opportunity to use the bus. After a device's transfer, the arbiter moves to the next requesting device.

Constraint Satisfaction

Constraint	**Solution**
6 USB peripherals	USB Host Controller
2 high-speed devices	Thunderbolt Controller
Avoid interrupt starvation	Interrupt queue + fair scheduling
Hardware interrupts	Device/controller IRQs
Software interrupts	OS system calls
Fair bus access	Round-robin arbitration

High-speed communication	Thunderbolt gets high-bandwidth PCIe-based path

Conclusion

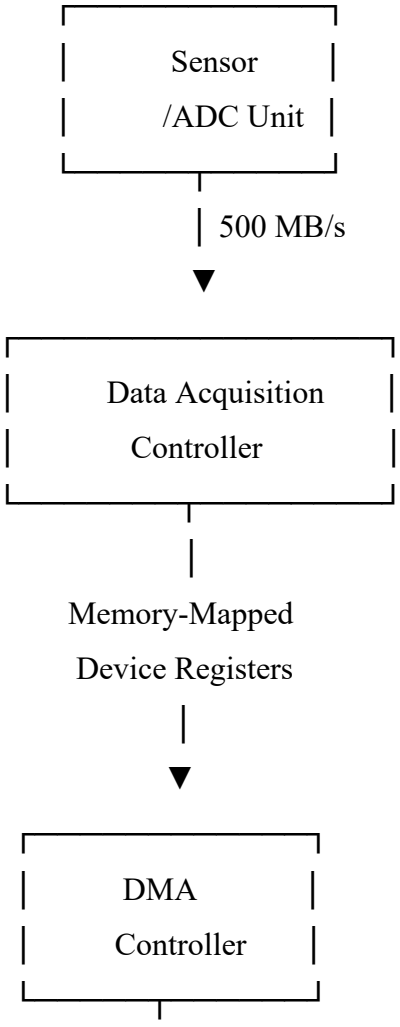
The architecture uses USB for six peripherals and Thunderbolt for two high-speed devices. Hardware interrupts provide fast device notification, while software interrupts support OS-level I/O services. Round-robin bus arbitration and interrupt queuing ensure that every device gets fair service and prevents interrupt starvation.

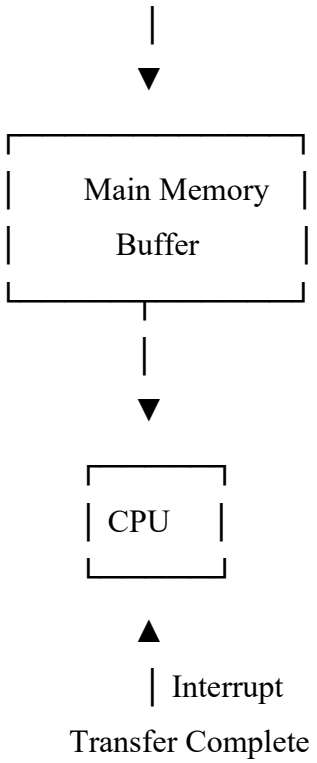
4. Data Acquisition System Design

Given Constraints

- Continuous data stream = 500 MB/s
- Minimal CPU intervention
- Compare DMA transfer and interrupt-driven I/O
- Use memory-mapped I/O for device registers

Proposed Architecture





Working

- The sensor/ADC continuously produces data at 500 MB/s.
- The device controller places data into a buffer.
- The CPU configures the DMA controller using memory-mapped registers.
- DMA transfers data directly from the acquisition device to main memory.
- The CPU is not involved in every byte or word transfer.
- DMA generates an interrupt after a block/buffer is transferred.
- The CPU processes the completed buffer while DMA fills the next buffer.

DMA vs Interrupt-Driven I/O

Feature	**DMA Transfer**	**Interrupt-Driven I/O**
CPU involvement	Very low	High
500 MB/s stream	Suitable	Not suitable
Data transfer	Direct device ↔ memory	CPU handles each transfer
Interrupt frequency	Low, per block	Very high
CPU overhead	Low	High
Efficiency	High	Low

Memory-Mapped I/O

The acquisition controller's registers are assigned memory addresses.

Example:

CONTROL_REG → Configure acquisition

STATUS_REG → Check device status

DMA_ADDR → Set memory address

DMA_SIZE → Set transfer size

The CPU accesses these registers using normal memory read/write instructions.

Best Choice

DMA is the better choice because a continuous 500 MB/s data stream would generate excessive CPU overhead if every data transfer required an interrupt.

A double-buffered DMA system is recommended:

Sensor → DMA → Buffer A → CPU processes

↓

Buffer B → CPU processes

While the CPU processes one buffer, DMA fills the other.

Conclusion

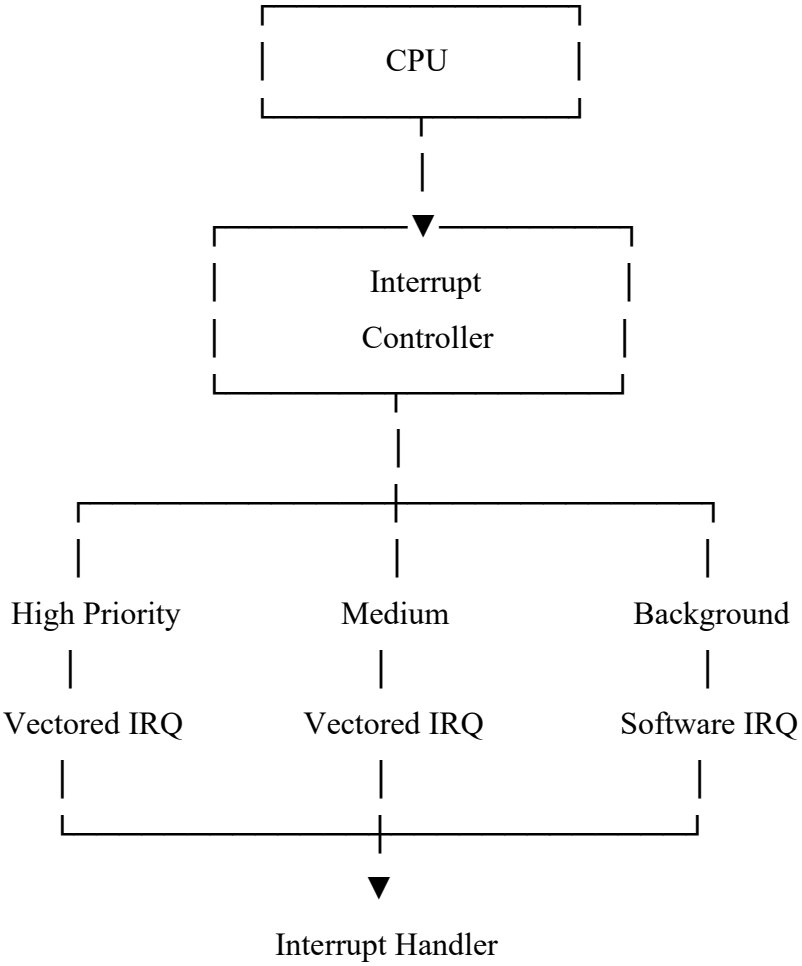
The recommended system is memory-mapped I/O + DMA + double buffering + block-completion interrupts. This provides continuous 500 MB/s data acquisition with minimal CPU intervention. DMA is clearly better than interrupt-driven I/O for this high-speed continuous data stream.

5. Interrupt Handling Mechanism

Given Constraints

- Support nested interrupts
- Maximum interrupt latency < 10 μs
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Proposed Architecture



Working

1. High-Priority Interrupts

Critical devices such as timers, communication interfaces, or emergency sensors use vectored interrupts. Each device has a predefined interrupt vector, allowing the CPU to immediately execute the correct ISR.

High-Priority Device

↓

Vector Number

↓

Correct ISR

↓

CPU

2. Nested Interrupts

The interrupt controller assigns priorities. While a low-priority ISR is executing, a higher-priority interrupt can interrupt it.

Low Priority ISR

↓

High Priority Interrupt

↓

High Priority ISR

↓

Return to Low Priority ISR

The CPU saves the current context before servicing the higher-priority interrupt.

3. Maximum Latency < 10 µs

To meet the latency requirement:

- Use vectored interrupts.
- Keep ISRs very short.
- Disable interrupts only for critical sections.
- Use a fast interrupt controller.
- Avoid lengthy processing inside ISRs.
- Move non-critical work to background tasks.

4. Software Interrupts

Software interrupts are used for non-critical/background operations such as:

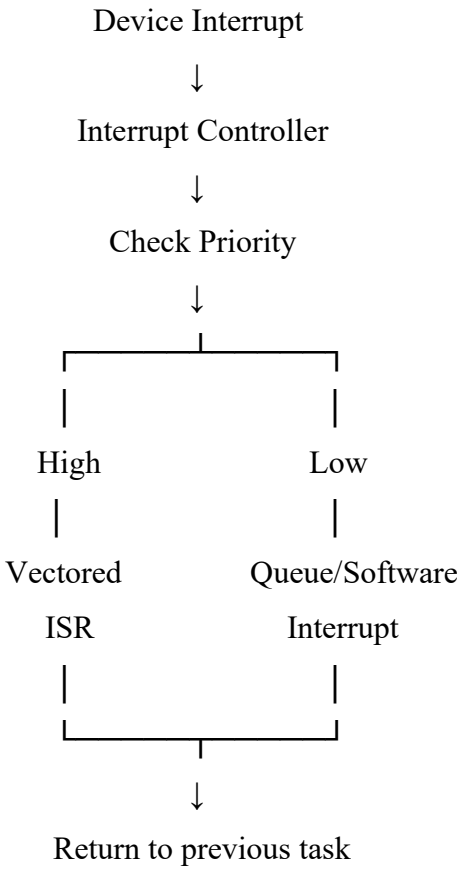
- Logging
- Data processing
- Maintenance tasks
- Operating-system services

These tasks do not interfere with high-priority real-time interrupts.

Priority Structure

Priority	**Interrupt Type**	**Example**	**Handling**
Highest	Hardware + vectored	Timer/emergency device	Immediate ISR
Medium	Hardware	Communication device	ISR
Lowest	Software interrupt	Logging/background work	Deferred processing

Example Flow



Conclusion

The best solution is a priority-based vectored interrupt system with nested interrupt support. High-priority devices are serviced immediately using vectored interrupts, while software interrupts handle background tasks. Short ISRs and controlled interrupt masking help maintain the required maximum interrupt latency of less than 10 μs.